EXPERIMENT - 6

Aim:-

Using the Employee schema, demonstrate:

- Stored Procedure transfer_employee(emp_id, new_dept_id) with validation and error handling
- Trigger for Salary Validation (INSERT and UPDATE)
- Triggers for Audit Logging (INSERT, UPDATE and DELETE)
- Testing of Edge Cases

Source Code:-

AUDIT TABLE

```
CREATE TABLE Employee_Audit
(
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT,
   action_type VARCHAR(20),
   old_dept_id INT,
   new_dept_id INT,
   old_salary DECIMAL(10,2),
   new_salary DECIMAL(10,2),
   changed_by VARCHAR(100),
   changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DESCRIBE Employee_Audit;
```

STORED PROCEDURE - transfer_employee

```
DELIMITER //

CREATE PROCEDURE transfer_employee
(
   IN p_emp_id INT,
   IN p_new_dept_id INT
)
BEGIN
   DECLARE v_old_dept INT;
   DECLARE v_count INT;
   DECLARE v_msg TEXT;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 v_msg = MESSAGE_TEXT;
      ROLLBACK;
      SELECT CONCAT('ERROR: ', v_msg) AS message;
   END;

   START TRANSACTION;
```

```
    -- Validation 1: employee must exist
    SELECT COUNT(*) INTO v_count
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;

    -- Validation 2: department must exist
    SELECT COUNT(*) INTO v_count
    FROM Department
    WHERE dept_id = p_new_dept_id;

    IF v_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Target department does not exist';
    END IF;

    -- Validation 3: employee must not already be in that department
    SELECT dept_id INTO v_old_dept
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_old_dept = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee is already in this department';
    END IF;

    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;

    COMMIT;

    SELECT CONCAT('SUCCESS: Employee ', p_emp_id,
    ' moved from department ', v_old_dept,
    ' to ', p_new_dept_id) AS message;
END //

DELIMITER ;
```

TRIGGER 1 - SALARY VALIDATION (BEFORE INSERT)

```
DELIMITER //

CREATE TRIGGER trg_salary_before_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
```

```
   IF NEW.salary IS NULL OR NEW.salary < 10000 OR NEW.salary > 500000 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid salary: must be between 10000 and 500000';
   END IF;
END //

DELIMITER ;
```

TRIGGER 2 - SALARY VALIDATION (BEFORE UPDATE)

```
DELIMITER //

CREATE TRIGGER trg_salary_before_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
   IF NEW.salary IS NULL OR NEW.salary < 10000 OR NEW.salary > 500000 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid salary: must be between 10000 and 500000';
   END IF;
END //

DELIMITER ;
```

TRIGGER 3 - AUDIT LOG (AFTER INSERT)

```
DELIMITER //

CREATE TRIGGER trg_audit_after_insert
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
   INSERT INTO Employee_Audit
   (emp_id, action_type, new_dept_id, new_salary, changed_by)
   VALUES
   (NEW.emp_id, 'INSERT', NEW.dept_id, NEW.salary, USER());
END //

DELIMITER ;
```

TRIGGER 4 - AUDIT LOG (AFTER UPDATE)

```
DELIMITER //

CREATE TRIGGER trg_audit_after_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
   IF NOT (OLD.salary <=> NEW.salary)
   OR NOT (OLD.dept_id <=> NEW.dept_id) THEN
      INSERT INTO Employee_Audit
      (emp_id, action_type, old_dept_id, new_dept_id,
       old_salary, new_salary, changed_by)
      VALUES
```

```
      (OLD.emp_id, 'UPDATE', OLD.dept_id, NEW.dept_id,
       OLD.salary, NEW.salary, USER());
   END IF;
END //

DELIMITER ;
```

TRIGGER 5 - AUDIT LOG (AFTER DELETE)

```
DELIMITER //

CREATE TRIGGER trg_audit_after_delete
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
   INSERT INTO Employee_Audit
   (emp_id, action_type, old_dept_id, old_salary, changed_by)
   VALUES
   (OLD.emp_id, 'DELETE', OLD.dept_id, OLD.salary, USER());
END //

DELIMITER ;
```

TEST 1 - VALID TRANSFER

```
CALL transfer_employee(1, 2);

SELECT emp_id, emp_name, dept_id
FROM Employee
WHERE emp_id = 1;

SELECT * FROM Employee_Audit;
```

TEST 2 - EDGE CASE: EMPLOYEE DOES NOT EXIST

```
CALL transfer_employee(999, 2);
```

TEST 3 - EDGE CASE: DEPARTMENT DOES NOT EXIST

```
CALL transfer_employee(1, 99);
```

TEST 4 - EDGE CASE: SAME DEPARTMENT

```
CALL transfer_employee(1, 2);
```

TEST 5 - EDGE CASE: NULL INPUT

```
CALL transfer_employee(NULL, 2);
```

TEST 6 - REVERT TRANSFER

```
CALL transfer_employee(1, 1);
```

TEST 7 - SALARY TRIGGER: INVALID UPDATES

```
UPDATE Employee SET salary = -5000 WHERE emp_id = 1;
```

```
UPDATE Employee SET salary = 5000 WHERE emp_id = 1;

UPDATE Employee SET salary = 600000 WHERE emp_id = 1;
```

TEST 8 - SALARY TRIGGER: VALID UPDATE AND AUDIT

```
UPDATE Employee SET salary = 70000 WHERE emp_id = 1;

UPDATE Employee SET salary = 65000 WHERE emp_id = 1;

SELECT * FROM Employee_Audit
ORDER BY audit_id;
```

TEST 9 - SALARY TRIGGER: INVALID INSERT

```
INSERT INTO Employee (emp_id, emp_name, dept_id, salary)
VALUES (101, 'Test Employee', 1, 0);
```

TEST 10 - AUDIT ON INSERT AND DELETE

```
INSERT INTO Employee (emp_id, emp_name, dept_id, salary)
VALUES (101, 'Test Employee', 1, 50000);

DELETE FROM Employee WHERE emp_id = 101;

SELECT * FROM Employee_Audit
ORDER BY audit_id;
```

SHOW TRIGGERS AND PROCEDURE

```
SELECT
   trigger_name,
   action_timing,
   event_manipulation,
   event_object_table
FROM information_schema.triggers
WHERE trigger_schema = DATABASE();

SELECT
   routine_name,
   routine_type
FROM information_schema.routines
WHERE routine_schema = DATABASE();
```

Output:-

AUDIT TABLE

Field	Type	Null	Key	Default	Extra
audit_id	int(11)	NO	PRI	NULL	auto_increment
emp_id	int(11)	YES		NULL	
action_type	varchar(20)	YES		NULL	
old_dept_id	int(11)	YES		NULL	
new_dept_id	int(11)	YES		NULL	
old_salary	decimal(10,2)	YES		NULL	
new_salary	decimal(10,2)	YES		NULL	
changed_by	varchar(100)	YES		NULL	
changed_at	timestamp	YES		current_timestamp()	

TEST 1 - VALID TRANSFER

message
SUCCESS: Employee 1 moved from department 1 to 2

emp_id	emp_name	dept_id
1	Aarav Sharma	2

audit_id	emp_id	action_type	old_dept_id	new_dept_id	old_salary	new_salary	changed_by	changed_at
1	1	UPDATE	1	2	65000.00	65000.00	root@localhost	2026-10-09 01:54:45

TEST 2 - EDGE CASE: EMPLOYEE DOES NOT EXIST

message
ERROR: Employee does not exist

TEST 3 - EDGE CASE: DEPARTMENT DOES NOT EXIST

message
ERROR: Target department does not exist

TEST 4 - EDGE CASE: SAME DEPARTMENT

message
ERROR: Employee is already in this department

TEST 5 - EDGE CASE: NULL INPUT

message
ERROR: Employee does not exist

TEST 6 - REVERT TRANSFER

message
SUCCESS: Employee 1 moved from department 2 to 1

TEST 7 - SALARY TRIGGER: INVALID UPDATES

ERROR 1644: Invalid salary: must be between 10000 and 500000

UPDATE Employee SET salary = -5000 WHERE emp_id = 1

ERROR 1644: Invalid salary: must be between 10000 and 500000

UPDATE Employee SET salary = 5000 WHERE emp_id = 1

ERROR 1644: Invalid salary: must be between 10000 and 500000

UPDATE Employee SET salary = 600000 WHERE emp_id = 1

TEST 8 - SALARY TRIGGER: VALID UPDATE AND AUDIT

UPDATE OK, 1 row(s) affected

UPDATE Employee SET salary = 70000 WHERE emp_id = 1

UPDATE OK, 1 row(s) affected

UPDATE Employee SET salary = 65000 WHERE emp_id = 1

audit_id	emp_id	action_type	old_dept_id	new_dept_id	old_salary	new_salary	changed_by	changed_at
1	1	UPDATE	1	2	65000.00	65000.00	root@localhost	2026-10-09 01:54:45
2	1	UPDATE	2	1	65000.00	65000.00	root@localhost	2026-10-09 01:54:45
3	1	UPDATE	1	1	65000.00	70000.00	root@localhost	2026-10-09 01:54:45
4	1	UPDATE	1	1	70000.00	65000.00	root@localhost	2026-10-09 01:54:45

TEST 9 - SALARY TRIGGER: INVALID INSERT

ERROR 1644: Invalid salary: must be between 10000 and 500000

INSERT INTO Employee (emp_id, emp_name, dept_id, salary) VALUES (101, 'Test Employee', 1, 0)

TEST 10 - AUDIT ON INSERT AND DELETE

INSERT OK, 1 row(s) affected

INSERT INTO Employee (emp_id, emp_name, dept_id, salary) VALUES (101, 'Test Employee', 1, 50...

DELETE OK, 1 row(s) affected

DELETE FROM Employee WHERE emp_id = 101

audit_id	emp_id	action_type	old_dept_id	new_dept_id	old_salary	new_salary	changed_by	changed_at
1	1	UPDATE	1	2	65000.00	65000.00	root@localhost	2026-10-09 01:54:45
2	1	UPDATE	2	1	65000.00	65000.00	root@localhost	2026-10-09 01:54:45
3	1	UPDATE	1	1	65000.00	70000.00	root@localhost	2026-10-09 01:54:45
4	1	UPDATE	1	1	70000.00	65000.00	root@localhost	2026-10-09 01:54:45
5	101	INSERT	NULL	1	NULL	50000.00	root@localhost	2026-10-09 01:54:45
6	101	DELETE	1	NULL	50000.00	NULL	root@localhost	2026-10-09 01:54:45

SHOW TRIGGERS AND PROCEDURE

trigger_name	action_timing	event_manipulation	event_object_table
trg_salary_before_insert	BEFORE	INSERT	Employee
trg_audit_after_insert	AFTER	INSERT	Employee
trg_salary_before_update	BEFORE	UPDATE	Employee
trg_audit_after_update	AFTER	UPDATE	Employee
trg_audit_after_delete	AFTER	DELETE	Employee

routine_name	routine_type
transfer_employee	PROCEDURE